WERKZEUG-KATALOG & TOOL-LANDSCHAFT

EDC15-DDE4 Toolbox & Steuergeräte-Werkzeuge

Vollständige Übersicht der eigenen Werkzeuge und der verifizierten GitHub-Landschaft für Bosch EDC15C4/DDE4, Siemens GS20 und Fahrzeug-Elektronik: Analyse, Diff, Transplantat, Prüfsummen, QA und Full-Flash.

Fahrzeug · Range Rover L322 TD6 · BMW M57 3.0d

Repo · github.com/batko15/EDC15-DDE4-Toolbox

Stand · 27. Oktober 2026

ANALYSE · TRANSPLANTAT · PRÜFSUMMEN · FULL-FLASH

Inhalt / Contents

1. Überblick: Was aufgebaut wurde **1**

2. Die eigene Toolbox: edc15toolbox.py **1**

3. GitHub-Katalog I: EDC15C4/DDE4-Kern & EDC15-Familie **2**

4. GitHub-Katalog II: Prüfsummen, Flashen & Protokolle **3**

5. GitHub-Katalog III: Diagnose, Editoren, CAN & Android **4**

6. Arbeitsablauf & Prüfsummen-Praxis **6**

7. Quellen, Foren & Rechtliches **6**

1. Überblick: Was aufgebaut wurde

Dieser Katalog bündelt den aktuellen Stand der Werkzeuglandschaft rund um den Range Rover L322 TD6 (BMW M57 3.0d, Bosch DDE4/EDC15C4) und die Siemens GS20-Getriebeesteuerung. Ausgangspunkt waren die Projekte der letzten Wochen: das Getriebetuning MAX3 FINALE, das DDE4-Map-Transplantat (330D 1600BAR in die RR-Original-Software 1037351777) und der STAGE8-Full-Flash mit vollständig reversierter Prüfsummen-Architektur. Die dabei entstandenen Skripte und Erkenntnisse sind jetzt in einem öffentlichen Repository konsolidiert, dokumentiert und als wiederverwendbares Kommandozeilen-Werkzeug verallgemeinert worden.

Ergänzend wurde die gesamte GitHub-Landschaft für Steuergeräte-Arbeit systematisch durchsucht: Flashen, Prüfsummen, Kennfeld-Bearbeitung, Diagnose, CAN-Bus und Android-Hardwarebrücken. Jeder einzelne Link in diesem Dokument wurde über die GitHub-API verifiziert — tote Repos sind ausgeschlossen, Sterne und Lizenzen entsprechen dem Stand vom 27.10.2026. Insgesamt wurden rund 70 Repositories geprüft und etwa 55 als relevant aufgenommen; die Maschinenlesbare Fassung des Katalogs liegt im Repo unter **docs/TOOL-KATALOG.md**.

Baustein	Beschreibung	Ort
EDC15-DDE4-Toolbox	Einheits-CLI (analyze, diff, records, slot-audit, port, verify, full-flash, sha256), Slot-Architektur-Doku, Workflow, GitHub-Katalog, Beispielprotokolle (VIN-maskiert)	https://github.com/batko15/EDC15-DDE4-Toolbox
QFLASH21	Komplette Diagnose- & Flash-App für DDE4: K-Line/KWP2000, 5-Baud-Init (Adresse 0x12, Keywords 45 05), Fehlerspeicher + Schattenspeicher, 51 DTC-Texte, 5 Messwertblöcke, Jobs, FULL/CAL-Lesen, CR2-Bankprüfsummen, Simulator, Android-APK (offline)	https://github.com/batko15/QFLASH21
Forschungsbasis	Zwei vorangegangene Deep-Research-Runden (Bosch EDC15C-Funktionsbeschreibung, Mursteinen-Dumps, DeepOBD-Format, Web-Serial-Fakten) liegen als Markdown im Repo: docs/RESEARCH/	docs/RESEARCH/

Tabelle 1: Die drei Säulen der Werkzeuglandschaft

2. Die eigene Toolbox: edc15toolbox.py

Das Herzstück der Erweiterung ist **edc15toolbox.py**: ein einziges Python-Skript ohne Abhängigkeiten (Python 3.8+), das die in den Projekten erprobte Logik zu acht Unterkommandos verallgemeinert. Jedes Unterkommando ist direkt aus der Praxis abgeleitet: Die Inventur- und Diff-Logik stammt aus dem Transplantat-Bau, der Slot-Audit aus der STAGE8-Verifikation, der Record-Parser aus der Karten-Analyse. Damit ist die gesamte Pipeline vom Roh-Dump bis zum geprüften Full-Flash in einem Werkzeug nachfahrbar — auf dem PC wie unter Termux auf dem Android-Telefon.

Kommando	Zweck
`analyze`	Datei identifizieren: 512K-Full-Flash vs 48K-Cal-Container, SW-Nummer (0xBFB0 Cal / 0x7BFB4 Boot-Read), Bosch-Strings, xxC1-Marker, Entropieprofil, SHA256

Kommando	Zweck
`diff`	Byte-Diff zweier BINs mit Regionen-Clustering (Block/Gap einstellbar), Hex-Vorschau, JSON-Export — Basis jeder Transplantat-Entscheidung
`records`	Record-Kette des DDE4-Cal-Containers parsen: , automatischer bester Startpunkt, Karten-/Kurven-Inventar als JSON
`slot-audit`	Prüfsummen-Slot-Audit (512K): Code-Kopf 0x13FFC, Kodierung 0x70B7C, Datenfenster 0x7BFFC; erkennt STALE-Zustand und verbietet Flashen bis zur Korrektur
`port`	Transplantat bauen: Spender-Maps ins Ziel, Schutz-Zonen + strukturellen Shift per JSON-Konfig (configs/), Build-Report für die QA
`verify`	QA gegen Original: Code/EWS/Fahrzeugdaten byteidentisch, Änderungen nur in Kopier-Regionen, Schutz-Zonen unberührt, optionale semantische Stichproben
`full-flash`	CAL-Container in 512K-Basis einbetten (Offset 0x70000) + SHA256SUMS schreiben
`sha256`	MD5/SHA256 von Dateien — Backup-Nachweis vor jedem Flash

Tabelle 2: Die acht Unterkommandos von edc15toolbox.py

Der Regressionstest ist der stärkste Qualitätshinweis: Mit der mitgelieferten Konfiguration **configs/l322_td6_1037351777_transplantA.json** reproduziert der generische port-Befehl das tatsächlich ausgelieferte L322-TD6-Transplantat **byteidentisch** (MD5 7843412d60754cb78ec66eb50fbac384, 10 868 geänderte Bytes in 175 Regionen, alle Schutz-Zonen unberührt). verify bestätigt danach ALLE PASS — die Generalisierung hat also nichts an der bewährten Logik verändert, sondern nur zugänglich gemacht.

Neben der CLI liegen im Repo die Original-Pipeline-Skripte (tools/l322_edc15c4/, tools/gs20_tcu/) als historische Referenz, die vendored GPL-3-Werkzeuge von moita420 (tdi41v2-Prüfsumme, C166-Disassembler, KWP1281-Probe) und von Mursteinen (EcuID) sowie die Projektprotokolle als Beispiele — die Voll-Flash-Protokolle mit VIN-Maskierung.

3. GitHub-Katalog I: EDC15C4/DDE4-Kern & EDC15-Familie

Diese Gruppe ist für den L322 TD6 unmittelbar einsatzbar. Absolute Kernreferenz ist das offizielle Bosch-Dokument „EDC 15C Funktionsbeschreibung“ (B079.CC0): Es beschreibt alle KWP2000-Services, den 28-Byte-Aufbau der Fehlerspeicher-Einträge, die EWS-Startwert-initialisierung (Routine 0x83), das 5-Baud-Init mit Keywords 6B/8F und das McMess-Alternativprotokoll. Zusammen mit den 23 echten ECU-Dumps der Mursteinen-Sammlung entsteht dadurch die wichtigste öffentliche Wissensbasis für genau dieses Steuergerät.

Repo	Sterne	Inhalt / Nutzen
`https://github.com/GabrielStanescu/BMW_M57_EDC15C4`	6	Bosch EDC15C Funktionsbeschreibung (PDF) + WinOLS-Projekte (damos/remap .ols) + Original-Flash 330d 184 PS (SW 1037351761)
`https://github.com/Mursteinen/BMW-DDE4-EDC15c4-EcuID`	2	23 echte EDC15C4-Dumps (BMW 4.0D, Mini, Rover 75, Freelander) + 2 IMMO-off-Boot-Diffs + SW-Nummer @0x7BFB4; GPL-3, vendored

Repo	Sterne	Inhalt / Nutzen
https://github.com/batko15/QFLASH21	0	DDE4-Diagnose & Flash (K-Line/KWP2000, PWA + Android-APK), CR2-Bankprüfsummen, Simulator

Tabelle 3: EDC15C4/DDE4-Kern

Repo	Sterne	Inhalt / Nutzen
https://github.com/moita420/edc15-tools	0	tdi41v2-Prüfsumme (EDC15V/VM/P), C166-Disassembler, KWP1281-Probe; GPL-3, vendored
https://github.com/mancube/EDC15M-Checksum-Corrector	k. A.	EDC15M „TDI V3.1“-Prüfsumme, CLI + GUI, Reverse-Engineering-Doku mitgeliefert
https://github.com/skaldamramra/VAGEDCSuite	35	Originale C#-Quelle des tdi41-Algorithmus (VAG-Suite, GPL-3)
https://github.com/NickTullos/edc15vm-flash-editor	6	.NET-7: EDC15V/VM-Flash-Inspektion mit automatischer DTC-Entdeckung — auf DDE4-Dumps übertragbar
https://github.com/krook1024/edc15-eoi	17	Einspritzbeginn-Rechner (EOI) für EDC15-Diesel
https://github.com/krook1024/edc15-swiss-knife	7	EDC15-Werkzeugkasten (Parser/Utilities)
https://github.com/krook1024/edcp	3	MSA15/EDC15-Datei-Parser
https://github.com/mtx-electronics/vag-checksum-fix	2	VAG-EDC15-Prüfsummen-Referenzpfad (Italien)
https://github.com/paceworx-store/EDC15C13-Checksum-Correction	0	EDC15C13-Prüfsumme — C-Schwester, Ansatz auf C4 testbar
https://github.com/marcelweinbach93-stack/VAGSUITE-NextGen	3	EDC15/16/17-Suite (junges Projekt, Entwicklung beobachten)
https://github.com/muki01/Bosch_EDC15_EEPROM_Tool	12	EEPROM-Tool (24C04): IMMO-Status, Login-Codes (VAG)
https://github.com/RJ-6463/Bosch_EDC15_EEPROM_Tool	4	Aktiver Fork des EEPROM-Tools
https://github.com/LeZed97/ZedSuite	175	Freier Kennfeld-Editor: beliebige BINs, Auto-Detection EDC15/16, 100 % lokal

Tabelle 4: EDC15-Familie allgemein (Methodik übertragbar)

4. GitHub-Katalog II: Prüfsummen, Flashen & Protokolle

Prüfsummen sind der Engpass jeder Überarbeitung: Erst wenn sie stimmen, ist eine Datei flashbar. Für die EDC15-Familien V/VM/P (tdi41v2) und M (TDI V3.1) existieren öffentliche, verifizierte Implementationen — beide im Repo verfügbar bzw. verlinkt. Für EDC15C4 gilt weiterhin: Es gibt kein öffentliches Korrekturmodul. Der Algorithmus liegt im maskierten ROM (Zielkonstante 0x4711B530, drei Prüf-Flows), alle Brute-Force-Anläufe gegen 46 Bilder blieben erfolglos. Der praktische Weg bleibt ein kommerzielles

WinOLS-Klasse-Modul, dessen Ergebnis danach zwingend mit slot-audit zu prüfen ist: Nach der Korrektur darf exakt nur 0x7BFFC (4 Bytes) geändert sein — alles andere ist Modul-Fehlverhalten.

Repo	Sterne	Inhalt / Nutzen
https://github.com/ConnorHowell/medc17-checksum-tool	59	Prüfsummen-Analyzer/Corrector für Bosch MED17/EDC17; MIT — Muster für eigene Analyzer
https://github.com/KWR666/Checksum-Corrector	0	MED7/MED9-Prüfsummen direkt per Handy — passt zur Android-Arbeitsweise
https://github.com/bri3d/VW_Flash	566	VW-UDS-Flashing: Kompression, Verschlüsselung, RSA-Bypass, Checksummen — Architektur-Referenz
https://github.com/ms4x-net/ms4x_flasher	27	MS4X Flasher: GS20/GS8.60.x/SMG2 lesen UND schreiben (unser Getriebe-Weg), lokale Checksum-Correction
https://github.com/dspl1236/VAGFlasher	28	Open-Source VAG ECU/TCU-Flasher (ME17/MED9/ME7/EDC17); MIT
https://github.com/DSparks156x/PQ35-Flasher	0	TP2/KWP2000-Flash, Readout und Diagnose für VAG PQ35
https://github.com/pandolfipedro/09G-Recovery-VW-AQ250-Flash	1	Aisin-09G-TCM-Recovery über KWP2000 — Wiederherstellungs-Referenz
https://github.com/aster94/Keyword-Protocol-2000	327	KWP2000-Arduino-Bibliothek: 5-Baud-Init/Timeout-Referenz-Implementation
https://github.com/muki01/OBD2_K-line_Reader	234	ISO9141/ISO14230-Reader für ESP32/Arduino (5-Baud + Fast-Init) — Hardware-Referenz
https://github.com/pylessard/python-udsoncan	734	UDS (ISO-14229) Python-Referenzimplementation; MIT
https://github.com/oxibus/automotive_diag	101	Rust-Crate: UDS/KWP2000/OBD-II/DoIP-Parser
https://pypi.org/project/isotp/	k. A.	ISO-TP (ISO-15765-2) Python-Modul (Doku: python-isotp.readthedocs.io)

Tabelle 5: Prüfsummen, Flashen & Protokoll-Bibliotheken

5. GitHub-Katalog III: Diagnose, Editoren, CAN & Android

Die Diagnose-Ebene ist bei BMW zweigeteilt: klassische E-Reihen laufen über EDIABAS/SGBD (DS2 bzw. KWP), neuere über CAN/UDS. uholeschak's ediabaslib ist die beste offene Implementation der EDIABAS-Logik und damit Referenz für alle SGBD-basierten Jobs. Für die Kennfeld-Arbeit bleibt TunerPro der Standard — unsere GS20-XDF-Definitionen sind dort entstanden; ms4x.net liefert die passenden Definition-Files und den Flasher. Die CAN-Sektion ist für tiefere Reverse-Arbeit (Hintergrundmonitor, CAN-Timeouts nach Flash-Sessions).

Repo / Quelle	Sterne	Inhalt / Nutzen
https://github.com/uholeschak/ediabaslib	939	.NET BMW/VAG-EDIABAS-Interpreter (DS2/DS1/KWP) — SGBD-Referenz
https://www.holeschak.de/BmwDeepObd/	k. A.	Deep OBD for BMW (App); Konfig-Format .ccpage/.cccfg/.ccpages vollständig per XSD dokumentiert
https://github.com/Dekon01/DeepObdE46Config	0	BmwDeepObd.xsd + E46-Beispielseiten (Benziner — Struktur-Referenz)
https://github.com/kmalinich/deepobd-configs	9	DeepOBD-XML-Konfigurationssammlung
https://github.com/tooming/garagediag	1	K-Line-Diagnose alte BMWs (E39/E87) über K+DCAN; MIT
https://github.com/dader34/BMWeb	18	BMW-Software-Tools-Sammlung (Windows); GPL-3
https://github.com/0xDevDav/EWSync	3	BMW EWS/DME-ISN-Synchronisation über DS2/K-Line; MIT — relevant für EWS-Themen
https://github.com/BimmerDock/ESeriesSwitch	1	One-Click-Umschalter für E-Reihen-Codier-Tools; MIT
https://www.tunerpro.net	k. A.	TunerPro 5 + XDF — Standard-Editor für GS20-/EDC15-Arbeit (nicht GitHub, gratis)
https://www.ms4x.net	k. A.	MS4X-Wiki + Definition Files (GS20-XDF) + ms4x_flasher
https://oldskulltuning.com/news/	k. A.	Laufend neue XDF-Definitionen (ECU + TCU) für TunerPro
https://github.com/RomRaider/RomRaider	373	Open-Source-Tuning-Suite (Editor + Logger) — Architektur-Referenz
https://www.evc.de	k. A.	WinOLS (kommerziell) — aktuell Pfad für EDC15C4-Prüfsummen
https://www.alientech-tools.com	k. A.	ECM Titanium (kommerziell, Damos-DB)
https://www.swiftec.org	k. A.	Swiftec (kommerziell, MAF/EGR/DTC-Module)
https://github.com/iDoka/awesome-canbus	3525	Awesome CAN-Bus: Tools, Hardware, Reverse-Engineering; CC0
https://github.com/jaredthecoder/awesome-vehicle-security	4729	Awesome Vehicle Security mit großem ECU-Kapitel
https://github.com/cantools/cantools	2305	DBC/ARXML-Kodieren/Dekodieren mit CLI; MIT
https://github.com/linux-can/can-utils	2941	SocketCAN-Standardwerkzeuge
https://github.com/collin80/SavvyCAN	1891	Qt-CAN-Sniffer mit Reverse-Engineering-Werkzeugen
https://github.com/commaai/panda	1707	Universal-CAN-Hardware + Firmware
https://github.com/cedricp/ddt4all	1909	Multi-Marken-Diagnose (ELM327)

Repo / Quelle	Sterne	Inhalt / Nutzen
https://github.com/brendan-w/python-OBD	1324	OBD-II-Python-Modul
https://github.com/rnd-ash/OpenVehicleDiag	1020	Rust-ECU-Diagnose (PassThru/J2534)
https://github.com/SchoepsLabs/webserial-android	7	Web Serial/WebUSB für Android-Geräte ohne Chrome 148 — Fallback für QFLASH21
https://github.com/selevo/WebUsbSerialTerminal	16	WebUSB-CH340-Terminal direkt im Android-Browser

Tabelle 6: Diagnose, Editoren, CAN-Bus & Android-Brücken

6. Arbeitsablauf & Prüfsummen-Praxis

Der erprobte Ablauf vom Roh-Dump bis zum flashfähigen Full-Flash besteht aus sieben Schritten, die vollständig mit der Toolbox nachfahrbar sind. Die Reihenfolge ist bewusst konservativ: Kein Schritt überspringt die QA, und nichts wird geflasht, bevor das Slot-Audit die Flashbereitschaft nicht bestätigt hat. Der Rückweg ist zu jedem Zeitpunkt das unveränderte Original-Backup aus Schritt 0.

```
python3 edc15toolbox.py sha256 BACKUP_ORIGINAL.bin
python3 edc15toolbox.py analyze DATEI.bin
python3 edc15toolbox.py diff ZIEL.bin SPENDER.bin --json diff.json
python3 edc15toolbox.py records CAL_48K.bin --json records.json
python3 edc15toolbox.py port ZIEL.bin SPENDER.bin --map configs/...json --out T.bin --report
r.json
python3 edc15toolbox.py verify ZIEL.bin T.bin --report r.json
python3 edc15toolbox.py slot-audit T.bin ZIEL.bin # vor Korrektur: slot2 STALE
# ... kommerzielles EDC15C4-Modul laufen lassen ...
python3 edc15toolbox.py slot-audit T_KORRIGIERT.bin T.bin # nur 0x7BFFC darf differieren
```

Slot	Adresse	Region	Regel
Code-Kopf	0x13FFC	[0x10000, 0x14000)	unverändert lassen (Code bleibt unberührt)
Kodierung	0x70B7C	[0x70000, 0x70B80)	unverändert lassen (Kodierbereich bleibt unberührt)
Datenfenster	0x7BFFC	[0x70B80, 0x7C000)	muss nach jeder Datenänderung vom Prüfsummenmodul neu gesetzt werden

Tabelle 7: Die drei Prüfsummen-Slots der DDE4 (bewiesen über 24 Referenz-Dumps)

Zwei harte Regeln haben sich in der Praxis bewährt. Erstens: Vor der Korrektur meldet das Slot-Audit den STALE-Zustand (slot2 noch Werkswert) — diese Datei darf niemals geflasht werden. Zweitens: Nach der Korrektur muss das Audit „PERFECT“ melden — exakt vier Bytes an 0x7BFFC differieren, sonst hat das Modul woanders etwas verändert und die Datei ist verworfen. Beide Prüfungen sind im Werkzeug verdrahtet und verlassen sich nicht auf Bildschirmausgaben externer Tools.

7. Quellen, Foren & Rechtliches

Quelle	Inhalt
https://github.com/GabrielStanescu/BMW_M57_EDC15C4	Bosch „EDC 15C Funktionsbeschreibung“ B079.CC0 (PDF): Services, Fehlerspeicher-Aufbau, EWS-Logik, 5-Baud-Init, McMess
https://ecuconnections.com	Tuning-Reverse-Engineering-Forum: EDC15-Threads, Prüfsummen-Diskussionen
https://www.e90post.com/forums/showthread.php?t=1204192	BMW Standard Tools 2.12 — Setup-DIY für INPA/NCS Expert/WinKFP/Tool32 (XP bis Win11)
https://bimmerpro.de/en/products/bmw-standard-tools-inpa-ncs-expert-download	BMW Standard Tools v2.12.8 (23 Tools) als gebündelter Download
https://www.rangerovers.net/forums	L322-TD6-Community: ECU-/Flash-Erfahrungen, Fahrzeugseiten
https://forum.tunerpro.net	TunerPro-Forum: XDF-Definitionen, Bereich „Andere Marken“ für BMW-Diesel

Tabelle 8: Wissensquellen & Foren

Rechtlicher Rahmen: Alle Werkzeuge sind für Forschung, Bildung und eigene bzw. autorisierte Fahrzeuge gedacht. Abgasrelevante Änderungen können die Straßenverkehrszulassung berühren; HU/TÜV-Lage und lokale Gesetze sind vor jedem Eingriff zu prüfen. Das Repository enthält bewusst keine Bosch-Firmware, keine Kalibrierdaten und keine Fahrzeug-BINs — die .gitignore sperrt alle binären Formate (*.bin, *.0DA, *.0PA). Das Bosch-PDF wird ausschließlich über den öffentlichen GitHub-Link des Original-Repositories referenziert, nicht kopiert.

Sicherheitshinweis zu den Zugangsdaten: Da die GitHub-Tokens im Chatverlauf im Klartext vorgelegen haben, sollten beide Tokens nach dieser Session in den GitHub-Einstellungen rotiert (widerrufen und neu erzeugt) werden — Feineinstellungen unter Settings → Developer settings → Personal access tokens. Das Repo selbst speichert keinerlei Zugangsdaten; der Push lief über einen temporären Remote-Eintrag, der nach dem Push wieder entfernt wurde.